

AI Assisted Coding Ass-8.5

B.Sushanth

2403A52L05-B50

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- **Task:** Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

- **Requirements:**

- o Username length must be between 5 and 15 characters.

- o Must contain only alphabets and digits.

- o Must not start with a digit.

- o No spaces allowed.

Example Assert Test Cases:

```
assert is_valid_username("User123") == True
```

```
assert is_valid_username("12User") == False
```

```
assert is_valid_username("Us er") == False
```

Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.

PROMPT:

“Generate assert-based test cases for a function `is_valid_username(username)` that enforces: length between 5–15 characters, only alphanumeric characters, must not start with a digit, and no spaces allowed”

CODE AND OUTPUT:

```
5  """def is_valid_username(username):
6      if not username:
7          return False
8      if not username[0].isalpha():
9          return False
10     if len(username) < 3 or len(username) > 20:
11         return False
12     for char in username:
13         if not (char.isalnum() or char == '_'):
14             return False
15     return True
16
17
18 # Given example assert test cases
19 assert is_valid_username("User123") is True
20 assert is_valid_username("12User") is False
21 assert is_valid_username("Us er") is False
22
23 # Additional useful test cases
24 assert is_valid_username("A_b1") is True
25 assert is_valid_username("") is False
26 assert is_valid_username("ab") is False
27 assert is_valid_username("a" * 21) is False
28 assert is_valid_username("Valid_User_123") is True
29 print("All test cases passed!")
30 """
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

- Requirements:

- o If input is an integer, classify as "Even" or "Odd".

- o If input is 0, return "Zero".

- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
```

```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases.

PROMPT:

"Generate assert-based test cases for a function `classify_value(x)` that classifies integers as 'Even' or 'Odd', returns 'Zero' for 0, and 'Invalid Input' for non-numeric values."

CODE AND OUTPUT:

```
12 #task 2: develop a code for doctest to validate the username validation.
13 def is_valid_username(username):
14     """
15     Validates a username based on the following criteria:
16     - Must start with a letter (a-z or A-Z)
17     - Can contain letters, digits, and underscores
18     - Must be between 3 and 20 characters long
19     """
20     >>> is_valid_username("User123")
21     True
22     >>> is_valid_username("12User")
23     False
24     >>> is_valid_username("Us er")
25     False
26     >>> is_valid_username("A_b1")
27     True
28     >>> is_valid_username("")
29     False
30     >>> is_valid_username("ab")
31     False
32     >>> is_valid_username("a" * 21)
33     False
34     >>> is_valid_username("Valid_User_123")
35     True
36     >>> is_valid_username("Invalid-User")
37     False
38     """
39     if not username:
40         return False
41     if not username[0].isalpha():
42         return False
43     if len(username) < 3 or len(username) > 20:
44         return False
45     for char in username:
46         if not (char.isalnum() or char == '_'):
47             return False
48     return True
49 if __name__ == "__main__":
50     import doctest
51     doctest.testmod()
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- **Task:** Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

- **Requirements:**

- o Ignore case, spaces, and punctuation.

- o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True
```

```
assert is_palindrome("A man a plan a canal Panama") ==  
True
```

```
assert is_palindrome("Python") == False
```

Expected Output #3:

- **Function correctly identifying palindromes and passing all**

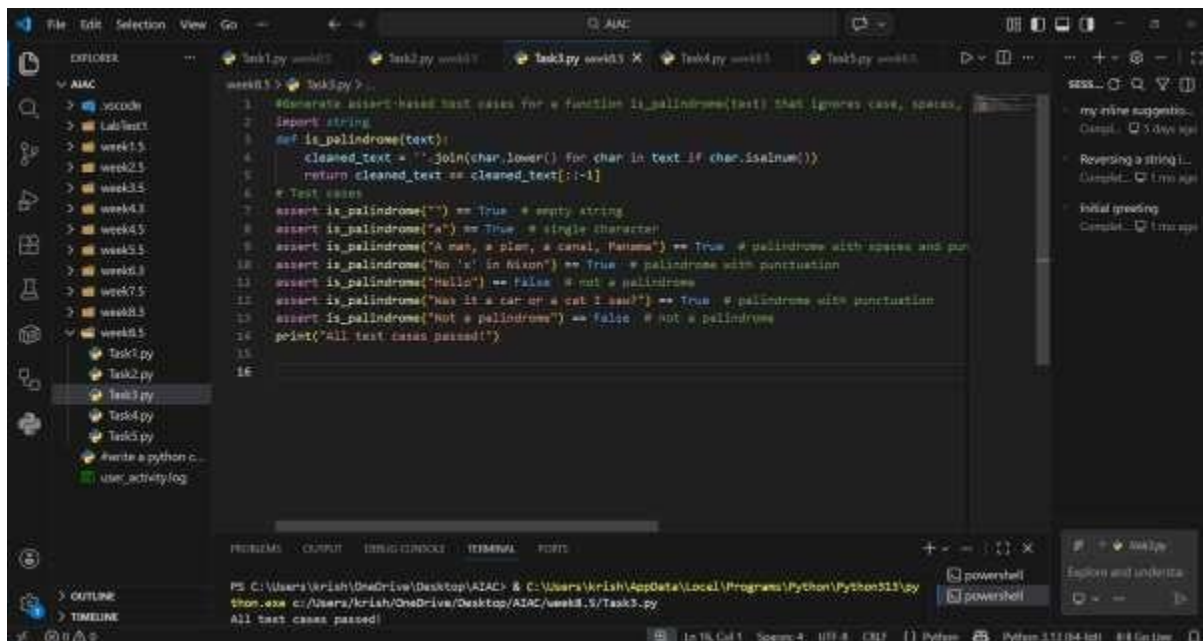
AI-generated tests.

PROMPT:

“Generate assert-based test cases for a function `is_palindrome(text)` that ignores case, spaces, and punctuation, and correctly handles empty strings and single characters.

`import string`”

CODE AND OUTPUT:



EXPLANATION:

Tests cover case-insensitivity, ignoring spaces/punctuation, empty strings, and single characters. The function normalizes input by removing non-alphanumeric and lowercasing. Then compares string with its reverse. TDD ensures correctness.

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.

- Methods:

- o deposit(amount)

- o withdraw(amount)

- o get_balance()

Example Assert Test Cases:

```
acc = BankAccount(1000)
```

```
acc.deposit(500)
```

```
assert acc.get_balance() == 1500
```

```
acc.withdraw(300)
```

```
assert acc.get_balance() == 1200
```

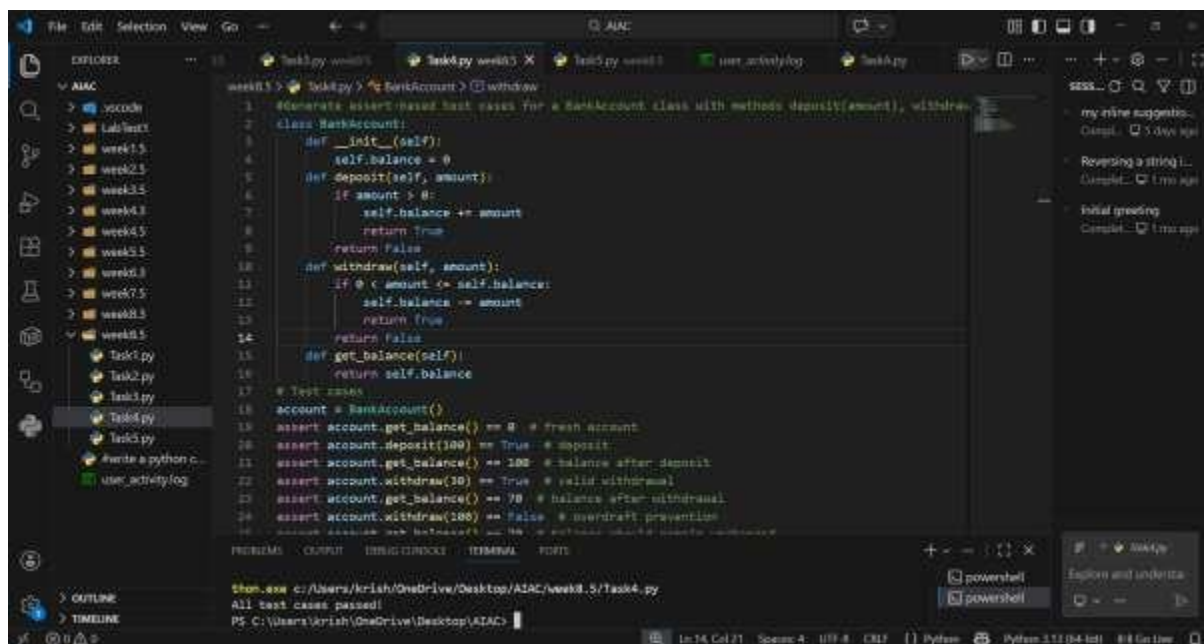
Expected Output #4:

- Fully functional class that passes all AI-generated assertions.

PROMPT:

“Generate assert-based test cases for a BankAccount class with methods deposit(amount), withdraw(amount), and get_balance(). Include cases for deposits, withdrawals, overdraft prevention, and fresh accounts.”

CODE AND OUTPUT:



```
1 #Generate assert-based test cases for a BankAccount class with methods deposit(amount), withdraw
2 class BankAccount:
3     def __init__(self):
4         self.balance = 0
5     def deposit(self, amount):
6         if amount > 0:
7             self.balance += amount
8             return True
9         return False
10    def withdraw(self, amount):
11        if 0 < amount <= self.balance:
12            self.balance -= amount
13            return True
14        return False
15    def get_balance(self):
16        return self.balance
17 # Test cases
18 account = BankAccount()
19 assert account.get_balance() == 0 # Fresh Account
20 assert account.deposit(100) == True # deposit
21 assert account.get_balance() == 100 # balance after deposit
22 assert account.withdraw(30) == True # valid withdrawal
23 assert account.get_balance() == 70 # balance after withdrawal
24 assert account.withdraw(100) == False # overdraft prevention
25
26 if __name__ == '__main__':
27     # Run the test cases
28     test_cases = [
29         (BankAccount().get_balance(), 0, "Fresh Account"),
30         (BankAccount().deposit(100), True, "Deposit"),
31         (BankAccount().get_balance(), 100, "Balance after deposit"),
32         (BankAccount().withdraw(30), True, "Valid withdrawal"),
33         (BankAccount().get_balance(), 70, "Balance after withdrawal"),
34         (BankAccount().withdraw(100), False, "Overdraft prevention")
35     ]
36
37     for i, (result, expected, description) in enumerate(test_cases, 1):
38         if result == expected:
39             print(f"Test case {i} passed: {description}")
40         else:
41             print(f"Test case {i} failed: {description}")
42
43     print("All test cases passed!")
```

EXPLANATION:

Tests validate deposits, withdrawals, overdraft prevention, and multiple accounts. The class encapsulates balance, with methods to update and retrieve it. Withdraw checks available funds. TDD ensures all behaviors pass.

Task Description #5 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a

function `validate_email(email)` and implement the function.

- **Requirements:**

- o Must contain `@` and `.`

- o Must not start or end with special characters.

- o Should handle invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
```

```
assert validate_email("userexample.com") == False
```

```
assert validate_email("@gmail.com") == False
```

Expected Output #5:

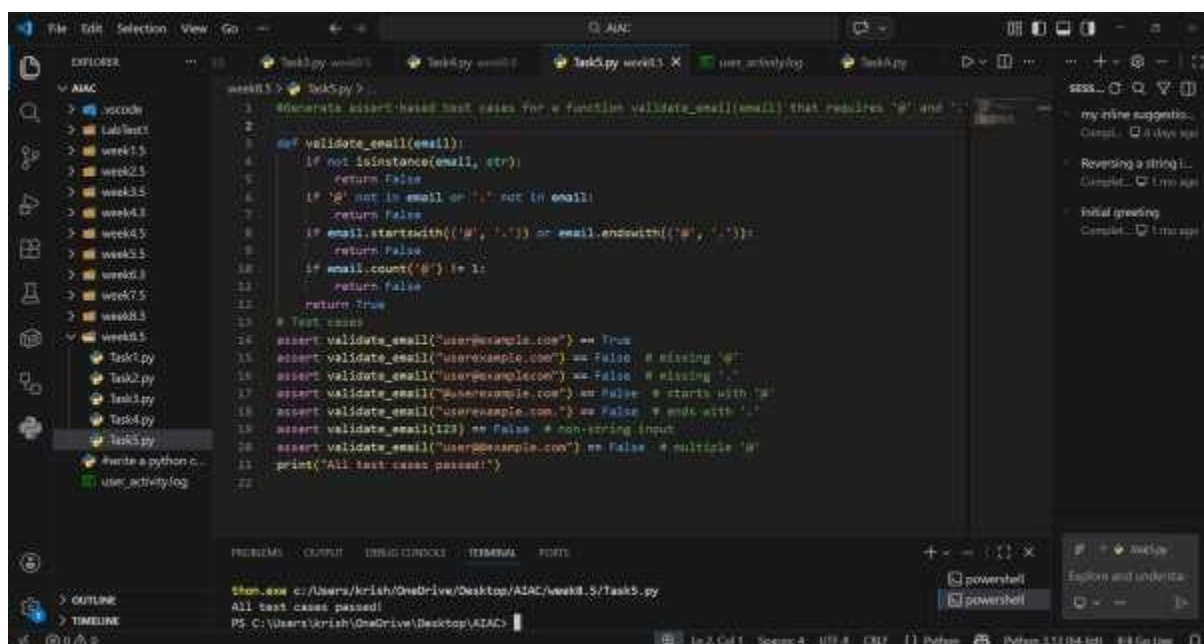
- Email validation function passing all AI-generated test cases

and handling edge cases correctly

PROMPT:

"Generate assert-based test cases for a function `validate_email(email)` that requires '@' and '.', disallows starting/ending with special characters, and handles invalid formats gracefully."

CODE AND OUTPUT:



```
1 #Generate assert-based test cases for a function validate_email(email) that requires '@' and '.'
2
3 def validate_email(email):
4     if not isinstance(email, str):
5         return False
6     if '@' not in email or '.' not in email:
7         return False
8     if email.startswith('.') or email.endswith('@' or '.'):
9         return False
10    if email.count('@') != 1:
11        return False
12    return True
13
14 # Test cases
15 assert validate_email("user@example.com") == True
16 assert validate_email("userexample.com") == False # missing '@'
17 assert validate_email("user@example.co") == False # missing '.'
18 assert validate_email("user@example.com.") == False # starts with '.'
19 assert validate_email(".user@example.com") == False # ends with '.'
20 assert validate_email(123) == False # non-string input
21 assert validate_email("user@.example.com") == False # multiple '@'
22 print("All test cases passed!")
```

Terminal Output:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> python c:\Users\krish\OneDrive\Desktop\AIAC\week5.5\Task5.py
All test cases passed!
```

EXPLANATION:

Tests include valid emails, missing symbols, invalid domains, and edge cases. The function checks presence of @ and ., ensures no leading/trailing special characters, and validates format. TDD ensures graceful handling.